

The Naylor Model

Variable Glossary

Reference for every column in the v3 / v4 / v5 model

Every variable has:

- a plain-English definition
- the exact formula and units
 - the data source
- the expected direction of effect on steal success

Speed & Acceleration

sprint_speed

Maximum foot-speed during the fastest 1-second window of a play.

Baseball Savant's 'Sprint Speed'. Computed only from 'competitive' runs (Statcast keeps the top ~2/3 of a player's sprints to remove easy jogs). Units: feet per second. League average is roughly 27.0 ft/s. Elite is 29+. Naylor sits around 24.4 ft/s, which is the ~3rd percentile of qualified MLB runners.

Source: `pybaseball.statcast_sprint_speed(year, min_opp=1)`

Expected effect: + on SB success (faster = harder to throw out)

speed_capped

sprint_speed clipped at 28 ft/s.

Empirically the marginal SB-success effect of speed flattens above ~28 ft/s. We confirmed this with a piecewise hinge regression: below-28 slope ≈ 0.21 (z-units), above-28 slope ≈ 0.06 . Capping at 28 prevents the model from over-rewarding speed once you are already 'fast enough', and lets variables like lead and jump matter more for elite-speed runners.

Source: `min(sprint_speed, 28.0)`

Expected effect: + but flat at the top

accel_0_30 (seconds to 30 ft)

Time, in seconds, to cover the first 30 feet from contact.

Pulled directly from Statcast running splits as `seconds_since_hit_030`. This is the cleanest single number for FIRST-BURST acceleration — the part of running that matters MOST for steals (you are only running ~90 ft and the throw is in the air long before you hit top speed).

League average ~1.78 s. Lower is better. Naylor 2025: 1.87 (slow on paper) — but he is fast in proportion to his small steps, see `accel_gap` below.

Source: `statcast_running_splits → seconds_since_hit_030`

Expected effect: - on SB success (less time = better)

accel_5_30

Time from 5 ft to 30 ft. Excludes the bat-flip/box variance.

Subtracts `seconds_since_hit_005` from `seconds_since_hit_030` so the metric is not contaminated by how quickly the runner cleared the batter's box. Pure 'first-burst once you are running' speed.

Source: `seconds_since_hit_030 - seconds_since_hit_005`

Expected effect: - on SB success

maintain_30_90

Time from 30 ft to 90 ft — top-speed maintenance.

Captures the second half of the sprint, where elite top-end matters. For an SB attempt the runner only goes ~85 ft, so this is partly a proxy for late-burst. Lower = better.

Source: `seconds_since_hit_090 - seconds_since_hit_030`

Expected effect: - on SB success

total_90

Total time, contact-to-90-ft (i.e. home-to-first).

Useful as a single 'how fast does this guy go a long way' number, but redundant with `accel_0_30` + `maintain_30_90`. League avg ~4.05 s.

Source: `seconds_since_hit_090`

Expected effect: - on SB success

accel_gap

Percentile rank of acceleration minus percentile rank of speed.

POSITIVE $\Rightarrow$ runner is faster off the line than their top speed implies. Naylor archetype.

Formula: `pctile(accel_0_30, inverted) - pctile(sprint_speed)`.

We invert `accel_0_30` because LOW time = HIGH percentile.

Range roughly -80 to +80 percentile points.

Source: Computed from `sprint_speed` + `accel_0_30` percentiles within season

Expected effect: + on SB success (independent of raw speed)

bolts

Number of plays in a season where the runner exceeded 30 ft/s.

Baseball Savant's 'Bolts' count. A simple count of elite-speed moments. Most slow runners have 0 bolts.

Source: Sprint-speed table → 'bolts' column

Expected effect: + on SB success

Lead Distance (real Baseball Savant data, v4)

primary_lead / lead_off_dist

Average distance off first base when the pitcher starts the motion.

The lead the runner takes BEFORE the pitcher commits. Real league average is roughly 11.5 ft.

Aggressive baserunners push 13+ ft; conservative is 10.5 ft.

In v3 this was a simulated draw $N(\text{lead_tendency_z}, 0.6)$.

In v4 we use the real Baseball Savant figure: `r_primary_lead`.

Source: baseballsavant.mlb.com/leaderboard/basestealing-run-value → `r_primary_lead` (real, 2015 onward)

Expected effect: + on SB success (longer head start)

secondary_lead

Average distance off first base at the moment of pitch release.

Primary lead PLUS the ground covered while the pitcher is delivering. League average ~14.5 ft. Elite secondary leads are 16+ ft.

Source: Baseball Savant → `r_secondary_lead`

Expected effect: + on SB success

lead_gain / r_sec_minus_prim_lead

DISTANCE COVERED FROM PITCHER'S FIRST MOVE TO PITCH RELEASE.

`secondary_lead - primary_lead`. This is the single best measure of what the user calls 'jerk' — how much ground the runner covered in the tiny window where the pitcher is committed but the ball is not yet released. Big `lead_gain` ⇒ the runner read the pitcher early AND had explosive first steps.

Naylor archetype lives here: his `lead_gain` is consistently >3.5 ft despite mediocre raw acceleration on a hit ball.

Source: Baseball Savant → `r_sec_minus_prim_lead`

Expected effect: + on SB success (strongly)

*_sbx variants (e.g. r_sec_minus_prim_lead_sbx)

Same metric, restricted to plays that ENDED in a SB attempt.

Baseball Savant publishes the lead metrics two ways: averaged over ALL plays the runner had on first (the base metric) and averaged only over plays where the runner went (the `_sbx` variant).

The `_sbx` version is cleaner for predicting SB success but smaller-sample.

Source: Baseball Savant

Expected effect: Same direction as base metric

Jump & Reaction

jump_time (derived)

Seconds from pitcher's first move to runner's break for second.

Not directly published by Statcast as a stand-alone leaderboard field. We approximate it by combining `lead_gain` with the running-splits acceleration profile: time to cover `lead_gain` feet from a standing start, given the runner's `accel_0_30`.

Lower = better jump. Range roughly 0.30 - 0.55 s.

Source: Derived from `r_sec_minus_prim_lead` + `statcast_running_splits`

Expected effect: - on SB success (less time = better)

reaction_quality

Composite of jump_time (low) + lead_gain (high).

v3 used a simulated $U(0.6, 0.95)$. v4 builds it from real components. A 0-1 score where 1 = optimal jump.

Source: v4: composite of real `jump_time` + `lead_gain`

Expected effect: + on SB success

Pitcher & Catcher Context

pitcher_ttp / pitcher_delivery_time

Pitcher Time-To-Plate: seconds from first motion to ball in glove.

Slow deliveries (≥ 1.40 s) give the runner more time. Fast deliveries (1.15 s slide-step) crush attempts. League avg ~ 1.30 s.

v3 SIMULATED this with $N(1.30, 0.10)$.

v4 DROPPED — couldn't be derived.

v5 USES LEAGUE CONSTANT 1.30 s. Real per-pitch delivery time is not publicly available; Savant's pitch-tempo CSV is buggy at source (median_seconds_empty equals median_seconds_onbase for every row, and year= filter is ignored).

Source: v5: league constant 1.30 s

Expected effect: + on SB success (longer TTP helps the runner)

pop_time_2b / pop_2b_sba

Pop time on stolen-base attempts to 2nd base (sec).

REAL per-catcher-per-year data (v5). From Savant's catcher pop time leaderboard. Elite catchers run pop times of 1.85 s and below. Bad pop times (2.05+) make stealing trivial. League avg ~ 1.95 s. Available 2018+ (not 2020).

Source: pybaseball.statcast_catcher_poptime(year, min_2b_att)

Expected effect: — on SB success (faster catcher = harder to steal)

exchange_2b_3b_sba

Catcher's glove-to-throwing-hand transfer time (sec).

Component of pop time. Real per-catcher (v5).

Source: pybaseball.statcast_catcher_poptime

Expected effect: — on SB success

maxeff_arm_2b_3b_sba

Catcher's max-effort arm strength on SB throws (mph).

Higher arm speed $\rightarrow$ faster throw $\rightarrow$ tighter window for steal.

Source: pybaseball.statcast_catcher_poptime

Expected effect: — on SB success

pitcher_lead_allowed_prim / sec / gain

Per-pitcher career: how much primary, secondary, lead-gain runners take.

From Savant pitcher-running-game CSV. Pitchers good at holding runners SHOW UP HERE with smaller lead values. Career-aggregate snapshot (year filter ignored). Cross-joined to each attempt by pitcher_id.

Source: <https://baseballsavant.mlb.com/leaderboard/pitcher-running-game>

Expected effect: + for runner side (more lead allowed = easier steal)

matchup_lead_diff

runner_lead_gain — pitcher_lead_allowed

Positive = this runner-pitcher matchup favors the runner. Negative = the pitcher's running-game suppression overcomes the runner's lead profile. v5 matchup feature.

Source: v5: derived

Expected effect: + on SB success

Real SB Performance & Residuals

SB / CS

Real stolen bases / caught stealing in the calendar season.

Hard counts from the MLB Stats API. We use them as ground truth. In v3 we required SB+CS ≥ 10 to qualify for the model.

Source: statsapi.mlb.com/api/v1/stats?stats=season&group=hitting

Expected effect: Ground-truth outcome

real_sb_pct

Shrunk Bayesian estimate of true SB% with $k=5$.

Formula: $(SB + k \cdot \text{LeagueSB\%}) / (SB + CS + k)$, $k = 5$.

Shrinkage prevents 1-for-1 runners from being scored as 100%.

League SB% is roughly 78% in the modern era.

Source: Computed from SB/CS

Expected effect: + proxy for skill (used as model target sometimes)

expected_sb_pct

Polynomial-predicted SB% given only the runner's sprint_speed.

We fit a 2nd-order polynomial of real_sb_pct on sprint_speed across all qualified runners.

This is the SB% you'd expect for a 'replacement-level baserunner' with the same speed.

Source: `np.polyfit(sprint_speed, real_sb_pct, deg=2)`

Expected effect: Baseline for residual

sb_residual

real_sb_pct - expected_sb_pct. THE key v3 signal.

Positive sb_residual $\Rightarrow$ the runner over-performs what their raw speed predicts. This is the empirical, speed-adjusted demonstrated steal skill. Naylor 2025: +0.108 (huge). Soto 2025: +0.105. This is what the SSSI is built around.

Source: `real_sb_pct - expected_sb_pct`

Expected effect: + direct measure of slow-steal skill

Situational Variables

late_game

Indicator: $\text{inning} \geq 7$.

Defenses may be more attentive late; runners may take fewer risks.

Source: Pitch-level data

Expected effect: Small - on attempts

outs

Number of outs (0/1/2) in the half-inning.

Two-out steals are riskier; managers send less.

Source: Pitch-level data

Expected effect: Ambiguous on success; - on attempt rate

p_throws_L, stand_L

Dummies: pitcher throws left / batter stands left.

Lefty pitchers face the runner directly $\rightarrow$ harder to steal.

Lefty batter blocks the catcher's throwing lane $\rightarrow$ slightly easier.

Source: Pitch-level data

Expected effect: p_throws_L: - stand_L: +

Compound Skill Indices (SSSI)

Z-score notation (suffix _z)

Any variable with _z is standardized to mean 0, SD 1.

Standardization happens within the qualified-runner pool. So `lead_off_z` = (lead off dist - pool mean) / pool SD. A value of +2 means '2 standard deviations above league'. This lets us add unlike variables (feet, seconds, percentiles) into one composite score.

Source: `sklearn.preprocessing.StandardScaler`

Expected effect: Same direction as un-standardized variable

SSSI_v3_fixed

Fixed-weight Slow-Steal Skill Index.

Plan-default linear combination of z-scores:

$$\begin{aligned} & 0.35 \cdot z(\text{sb_residual}) \\ & + 0.25 \cdot z(\text{accel_gap}) \\ & + 0.15 \cdot z(\text{lead_gain}) \\ & + 0.10 \cdot z(\text{reaction_quality}) \quad (= \text{'jump'}) \\ & + 0.10 \cdot z(\text{lead_off_dist}) \\ & - 0.05 \cdot z(\text{speed_capped}) \end{aligned}$$

Higher = better slow-steal archetype. Designed to reward skill independent of raw speed.

Source: Computed in v3 pipeline

Expected effect: + identifies slow-but-effective basestealers

SSSI_v3_opt (optimised)

Grid-searched weights that maximise Naylor+Soto z-score.

From 1,728 weight combinations on a 6-d grid we picked:

$$\begin{aligned} & 0.25 \cdot z(\text{sb_residual}) \\ & + 0.10 \cdot z(\text{accel_gap}) \\ & + 0.05 \cdot z(\text{lead_gain}) \\ & + 0.05 \cdot z(\text{reaction_quality}) \\ & + 0.20 \cdot z(\text{lead_off_dist}) \\ & - 0.20 \cdot z(\text{speed_capped}) \end{aligned}$$

Under these weights Naylor 2025 ranks #1 (SSSI = 2.10), Soto 2025 ranks #3 (SSSI = 1.61).

Source: Grid search inside v3 pipeline

Expected effect: Optimal for the slow-steal archetype

lead_tendency_z (latent variable)

Latent runner-specific tendency to take big leads.

In v3 this was a random $N(0,1)$ draw — except Naylor and Soto were anchored at +2.0 (documented elite leads). In v4 this is REPLACED with real $r_primary_lead$ z-score and the anchoring is removed.

Source: v3 simulated; v4 = $z(r_primary_lead)$

Expected effect: + on SB success

Per-Pitch Features (v5 only)

y_attempt / y_success

Binary labels per attempt: was there an attempt? did it succeed?

Parsed from the Statcast `des`` (description) text field. Regex matches 'steals (N) 2nd base' → SB; 'caught stealing 2nd' → CS; 'picks off ... at 1st' → PK. CRITICAL LIMITATION: the count, outs, and inning attached to an attempt are those of the at-bat's FINAL pitch (where the SB `des` was recorded), not the actual mid-at-bat pitch on which the SB happened.

Source: v5: regex on Statcast `des` field

Expected effect: Ground-truth outcome ($y_success$ is the target of Model A)

balls, strikes (count)

Pre-pitch count at the at-bat's final pitch (per-pitch field).

Real per-pitch from Statcast. Used to compute count-leverage (see `is_count_HL` below).

Source: Statcast

Expected effect: Context — interaction with attempt rate

count_label

String 'balls-strikes', e.g. '1-1', '3-2'.

Used to bucket pitches into the 12 possible counts.

Source: v5: derived

Expected effect: Categorical context

is_count_HL (high-leverage count flag)

1 if this count's attempt rate is $\geq 60\%$ of the peak count's rate.

Per user spec, low-attempt counts are DROPPED from per-attempt training because they add noise. The empirical HL set is computed at runtime; typical HL counts: 0-0, 1-0, 2-1, 1-1, 3-1.

Source: v5: derived from league attempt-rate table

Expected effect: Flag — used as a feature AND a training filter

pct_in_HL (per runner-season)

Fraction of a runner's attempts that occurred in HL counts.

Aggregate version of is_count_HL. Runners who pick their spots well will have high pct_in_HL.

Source: v5: derived

Expected effect: + on overall SB%

outs_when_up, inning, inning_topbot

Game-state at the moment of the at-bat (real per-pitch).

Outs: 0/1/2. Two-out steals are riskier; managers send less.

Inning: 1-15+. Late-game leverage influences attempt rate.

Top/bot: game half.

Source: Statcast per-pitch

Expected effect: Context

pre_rel_vel (NEW v5 metric)

Runner's avg velocity from pitcher's first move to release (ft/s).

Formula: $r_{\text{sec}} - \text{prim_lead} / \text{pitcher_delivery_proxy_s}$

$= \text{lead_gain (ft)} / 1.30 \text{ s (league-constant proxy)}$

Variation in this metric is driven ENTIRELY by lead_gain because we have no real per-pitcher delivery time. Still useful as a normalised distance-per-time metric.

User's intent: a runner who covers more ground in the same pitcher-delivery window is more dangerous, even against fast pitchers.

Source: v5: derived from runner career lead + league constant

Expected effect: + on SB success

post_rel_dist (NEW v5 metric)

Distance runner covers during catcher's pop window (ft).

Formula: $\text{sprint_speed} \times \text{pop_2b_sba} - \text{accel_correction}$

where $\text{accel_correction} = 0.5 \times \max(0, \text{accel_0_30} - 1.65)$

$\times \text{sprint_speed} \times \text{pop_time}$.

Slower-acceleration runners haven't reached top speed yet so the naive $\text{sprint} \times \text{pop}$ product over-counts. The correction scales the penalty with the runner's accel-time deficit.

User's intent: even a slow runner who covers a lot of ground during the pop window is dangerous on the back end.

Source: v5: derived from real per-attempt pop + runner profile

Expected effect: + on SB success

pre_rel_vel_avg, post_rel_dist_avg (per runner-season)

Per-attempt mean of the above two metrics over a season.

Aggregated for use in Model B (season-level GBM) and SSSI v5.

Source: v5: `groupby(runner_id, season).mean()`

Expected effect: + on overall SB%

avg_pop_faced

Average catcher pop time across this runner's SB attempts.

Real per-attempt catcher pop (v5) → mean per runner-season. Runners who attack weak-arm catchers will have HIGH avg_pop_faced.

Source: v5: derived from per-attempt join

Expected effect: + on overall SB%

seconds_since_hit_005 ... seconds_since_hit_090

18 raw Statcast running-splits columns (5/10/15/.../90 ft).

Each column is seconds elapsed from contact at that distance milestone. v5 explores three representations of this curve (see curve_a, curve_b, curve_c below). Lower = faster.

Source: `pybaseball.statcast_running_splits(year, raw_splits=True)`

Expected effect: — on SB success (less time = better)

curve_a, curve_b, curve_c, curve_rmse

Quadratic fit to runner's velocity curve over 5-90 ft.

Fits $\text{time} = a \cdot d^2 + b \cdot d + c$ to the 18 split points per runner-season.

a captures curvature (fatigue / late drag).

b is the linear time/distance slope ($\approx 1/\text{avg-speed}$).

c is the intercept (initial reaction time after contact).

rmse measures how well a quadratic fits the actual splits.

v5 'Path B' representation of the splits.

Source: v5: np.polyfit per row across 18 split columns

Expected effect: Combined ≈ 1 strong feature

SSSI_v5

v5 composite — adds pre_rel_vel_z and post_rel_dist_z to v4.

Linear combination of 8 z-scored terms. Weights are grid-searched on 80% of runners (Naylor and Soto are HELD OUT from the grid search to avoid overfitting them).

Source: Computed in v5_explore.py

Expected effect: + identifies slow-steal archetype with new metrics included